LEITURA TÉCNICA DE PROJETO

Projeto JUMO

Documento de apoio técnico sobre o aplicativo desenvolvido no repositório **Symbius** — um app de apoio entre estudantes construído em React Native, com Firebase como back-end.

Preparado como material de referência para elaboração de uma apresentação.
Foco: tecnologia e comportamento do produto, em linguagem acessível.
Data da leitura: 14 de setembro de 2026

SEÇÃO 1

Visão Geral do Projeto

O projeto lido é um aplicativo móvel chamado **JUMO**, criado para os estudantes das **Escolas Padre Anchieta**, em Jundiaí (SP). O código do app confirma isso diretamente: a tela de contato traz o endereço, telefone e WhatsApp reais da instituição, e um dos arquivos internos faz referência a uma "Especificação de Gamificação e Identidade Visual — TCC", o que indica que o projeto nasceu como um **Trabalho de Conclusão de Curso**.

O que o app propõe

O JUMO é, essencialmente, uma **plataforma de apoio entre estudantes**: alunos que têm facilidade em uma matéria podem ajudar colegas com dificuldade nela, através de um sistema de pareamento ("Match de Estudos") e de um chat próprio. Para incentivar o uso contínuo, o app tem uma camada de **gamificação** completa — pontos de experiência, moedas virtuais, níveis e uma lojinha de itens cosméticos — além de funcionalidades de apoio ao dia a dia escolar, como calendário de provas e eventos, mural de avisos e um painel administrativo para a gestão do conteúdo.

Como o app foi construído, em uma frase

É um aplicativo **React Native** (então funciona em Android, iOS e Web a partir de um único código-fonte), escrito em **TypeScript**, que usa o **Firebase** do Google como back-end — ou seja, não existe um servidor próprio guardando os dados: quem guarda, autentica os usuários e distribui as atualizações em tempo real é o Firebase.

React Native + Expo **TypeScript** **Firebase (Firestore + Auth)** **React Navigation** **Notificações (Expo)** **Armazenamento local (AsyncStorage)**

Observação: o nome do repositório no computador é "Symbius", enquanto o nome do aplicativo em si — visível para o usuário final, na loja de apps e nas configurações do projeto — é "JUMO". Ao longo deste documento, o produto é chamado pelo nome com que ele se apresenta: JUMO.

SEÇÃO 2

Como o Projeto Foi Construído

Esta seção explica, em ordem, as peças de tecnologia que sustentam o app e o papel de cada uma — sem entrar em código, apenas no "para que serve".

A base: React Native + Expo

O aplicativo é escrito em **React Native**, uma tecnologia que permite construir apps de celular usando os mesmos conceitos do React (popular para sites), mas gerando telas nativas de verdade — não uma página web dentro de um app. Isso quer dizer que a mesma base de código do JUMO roda em **Android**, em **iPhone (iOS)** e até em **navegador (Web)**, sem reescrever o app três vezes.

Sobre o React Native, o projeto usa o **Expo** (mais especificamente, a versão 57 do seu conjunto de ferramentas). O Expo é uma camada que facilita muito a vida de quem desenvolve: cuida da configuração inicial do projeto, oferece prontos recursos comuns de celular (câmera, notificações, fontes, etc. — todos usados no JUMO) e simplifica o processo de gerar o instalador final do app. Para um projeto acadêmico com prazo, essa escolha reduz bastante o tempo gasto com configuração e infraestrutura, deixando mais tempo para as funcionalidades.

A linguagem: TypeScript

O código é escrito em **TypeScript**, e não em JavaScript puro. Na prática, TypeScript é o JavaScript de sempre, com um adicional: cada informação (nome, idade, lista de mensagens etc.) tem um "tipo" declarado. Isso faz com que muitos erros comuns de programação — como tentar usar um texto no lugar de um número, ou esquecer um campo obrigatório — sejam apontados antes mesmo do app rodar, direto no editor de código. O projeto usa inclusive o modo "strict" (mais rigoroso) do TypeScript, o que é considerado uma boa prática de mercado para projetos que devem crescer com segurança.

Navegação entre telas

A troca entre as telas do app (Login → Início → Chat → Perfil, etc.) é feita com a biblioteca **React Navigation**, o padrão de fato do ecossistema React Native. É ela quem organiza a "pilha" de telas, controla o botão de voltar e permite passar informação de uma tela para a outra (por exemplo, abrir o Chat já sabendo com qual colega a conversa é).

Organização do estado da aplicação

Informações que muitas telas precisam conhecer ao mesmo tempo — como o tema claro/escuro escolhido, o tamanho da fonte, ou se há uma notificação nova de chat — são centralizadas em dois "provedores de contexto" (*Context API*, um recurso nativo do próprio React): um cuida das preferências do usuário, e outro cuida de toda a lógica de notificações em tempo real. Essa é uma escolha enxuta: para o tamanho deste app, não foi necessário adotar uma biblioteca externa de gerenciamento de estado (como Redux), o que mantém o projeto mais simples de manter.

Identidade visual e fontes

O app tem uma paleta de cores e tipografia próprias, centralizadas em um único arquivo de configuração — o que facilita manter a aparência consistente em todas as telas e também é o que possibilita o **modo escuro**, já implementado. A fonte utilizada é a **Poppins** (carregada via Google Fonts), em cinco pesos diferentes.

SEÇÃO 3

O Firebase e Por Que Ele Se Encaixa

Esta é provavelmente a decisão técnica mais importante do projeto, então vale explicar com calma — inclusive para quem nunca ouviu falar em Firebase.

O que é o Firebase, em termos simples

> **Analogia**
>
> Pense no Firebase como um "prédio de infraestrutura pronto" oferecido pelo Google: em vez de a equipe construir e manter seu próprio servidor, seu próprio sistema de login e seu próprio banco de dados do zero, ela aluga essa infraestrutura já pronta, testada e que escala sozinha conforme o número de usuários cresce.

Esse modelo é chamado de **Backend as a Service (BaaS)** — "back-end como serviço". O JUMO usa duas partes específicas do Firebase:

1) Firestore — o banco de dados

O **Firestore** é o banco de dados do projeto. Ele guarda tudo: os usuários cadastrados, as mensagens do chat, os posts do mural, os eventos do calendário, as solicitações de match, os pontos de gamificação etc. A característica mais importante do Firestore, e a que mais influencia o comportamento do JUMO, é que ele é um banco de dados **em tempo real**: quando algo muda no banco, qualquer tela aberta que dependa daquela informação é atualizada sozinha, sem precisar recarregar a página ou apertar "atualizar".

2) Firebase Authentication — o login

A **Firebase Authentication** cuida de todo o processo de criar contas e fazer login com e-mail e senha, incluindo o envio automático de e-mail de verificação (usado no cadastro do JUMO para confirmar que o e-mail escolar informado é válido antes de liberar o acesso).

Por que essa escolha faz sentido para este projeto

Motivo	Por que importa aqui
Tempo real "de fábrica"	O comportamento mais visível do JUMO — chat instantâneo, aviso de "está online", banner de nova mensagem, notificação de novo match — depende de saber, na hora, que algo mudou. É exatamente isso que o Firestore entrega pronto, sem a equipe precisar construir esse mecanismo do zero.
Sem servidor próprio para manter	Não é preciso alugar, configurar ou monitorar um servidor. Isso é especialmente relevante em um projeto acadêmico, onde tempo e equipe são limitados.
Integração direta com React Native	O Firebase tem um kit oficial para este tipo de app, com boa documentação e grande comunidade — o que reduz a curva de aprendizado.
Escala automaticamente	Do primeiro teste com poucos alunos até um uso maior da escola inteira, a infraestrutura acompanha sem mudança de arquitetura.
Plano gratuito generoso	Para o volume de uma escola (uso do projeto atual), o Firebase pode operar inteiramente dentro da camada gratuita, o que é ideal para um projeto acadêmico sem orçamento de infraestrutura.

Um ponto técnico que vale mencionar

O Firebase também oferece um serviço próprio de armazenamento de arquivos (o "Firebase Storage"), pensado especificamente para guardar fotos, vídeos e documentos. Na leitura do código, porém, o JUMO não usa esse serviço: as imagens, vídeos e documentos enviados no chat, nos posts e nas fotos de perfil são convertidos em texto (formato Base64) e guardados *dentro* dos próprios documentos do Firestore. Isso funciona no estágio atual do projeto, mas é um ponto relevante para uma futura etapa de melhoria de desempenho — o Firestore não foi desenhado para guardar arquivos grandes dessa forma, e isso tende a pesar conforme o volume de mídia cresce. Fica como contexto, sem entrar em correções agora, já que este documento é apenas a leitura técnica.

SEÇÃO 4

Outras Tecnologias de Apoio

Além da dupla React Native + Firebase, o projeto reúne um conjunto de bibliotecas menores, cada uma resolvendo um problema pontual. Nenhuma delas é o "coração" do app, mas juntas explicam boa parte do comportamento visível.

Tecnologia	Categoria	Para que é usada no JUMO
AsyncStorage	Armazenamento local	Guarda no próprio celular a sessão do usuário (para não precisar logar toda vez) e preferências como tema e tamanho de fonte, que não precisam depender de internet.
Expo Notifications	Notificações	Mostra notificações do sistema (mensagem nova, novo pedido de match, novo evento) mesmo com o app em segundo plano, e trata o toque na notificação para abrir a tela correta.
React Navigation	Navegação	Controla a troca entre as telas do app.
Expo Image Picker	Mídia	Permite escolher fotos da galeria (foto de perfil, capa de post, imagens no chat), já com compressão para reduzir o tamanho do arquivo.
React Native Chart Kit	Visualização de dados	Gera os gráficos (linhas, barras, pizza) usados no painel administrativo.
date-fns	Datas	Cálculos e formatação de datas usados no calendário de eventos e provas.
Google Fonts (Poppins)	Identidade visual	Tipografia oficial usada em todo o app.
EAS (Expo Application Services)	Build e distribuição	Serviço usado para gerar os instaladores do app (ex.: arquivo .apk para testes internos) e, futuramente, publicá-lo nas lojas de aplicativos.

Duas peças que aparecem no projeto, mas merecem atenção à parte

Um servidor próprio de chat (server.js): o projeto inclui um pequeno script Node.js que sobe um servidor de "WebSocket" (uma tecnologia de comunicação instantânea entre cliente e servidor) na porta 3001. Ao conferir o restante do código do app, não foi encontrada nenhuma tela se conectando a esse servidor — o chat em tempo real do JUMO, como está hoje, funciona inteiramente através dos "escutadores" em tempo real do Firestore, descritos na Seção 3. Ou seja: esse script parece ser uma peça auxiliar (possivelmente de uma etapa anterior do projeto, ou reservada para uso futuro) que não está conectada ao app atualmente. Vale confirmar isso com quem desenvolveu o projeto antes de mencioná-lo na apresentação.

EmailJS: essa biblioteca (que permite enviar e-mails direto do app, sem servidor próprio) está listada entre as dependências do projeto, mas não foi encontrada em uso em nenhuma tela — a tela de Contato, por exemplo, usa apenas links diretos para WhatsApp, telefone, e-mail e site da escola. Também parece ser algo planejado ou de uma versão anterior do app.

SEÇÃO 5

Como o App se Comporta — Tela por Tela

Esta seção descreve o que cada tela **faz** do ponto de vista de quem usa o app — é a parte mais útil para explicar o projeto a quem não é da área técnica, já que fala de comportamento e não de código.

Entrada no app

CADASTRO E LOGIN

Cadastro → Login

O aluno se cadastra informando nome, RA (registro do aluno), turma e o tipo de perfil. A conta nasce com status "pendente" e só é liberada depois que o e-mail é verificado — um clique no link enviado automaticamente pelo Firebase. No login seguinte, o app lembra a sessão salva no celular e pula a tela de login sempre que possível, indo direto para a Página Inicial.

O dia a dia do estudante

PÁGINA INICIAL

Início (Index)

É a tela central do app: reúne o mural de posts/avisos, os lembretes pessoais do aluno e os eventos oficiais da escola relevantes para a série dele, além de dar acesso rápido às demais áreas (chat, match, calendário, loja, perfil).

PAREAMENTO ENTRE ALUNOS

Match de Estudos

É o coração do propósito do app. Cada aluno indica em quais matérias (Português, Matemática, Física, Química, etc.) ele pode ajudar outros colegas e em quais ele gostaria de receber ajuda. A partir disso, é possível descobrir colegas compatíveis, enviar um pedido de "match", e — se aceito — uma conversa é aberta automaticamente entre os dois. Ao final do apoio, quem recebeu ajuda avalia (de 0 a 5 estrelas) se a dúvida foi resolvida, e essa avaliação define quanta recompensa de gamificação o colega que ajudou recebe.

CONVERSA EM TEMPO REAL

Chat

Conversas individuais com texto, fotos, vídeos e documentos, indicador de "online" e "visto por último", e um banner que aparece na tela assim que uma mensagem nova chega — tudo isso atualizado instantaneamente através do Firestore, sem precisar atualizar a tela manualmente.

ORGANIZAÇÃO ESCOLAR

Calendário

Um calendário único que combina os eventos oficiais da escola (provas, feriados, avisos e eventos gerais, filtrados pela série do aluno) com lembretes pessoais que o próprio aluno pode criar para si mesmo.

A camada de engajamento (gamificação)

EVOLUÇÃO DO ALUNO

Sistema de Gamificação

Toda interação positiva no app — fazer check-in diário, concluir um match de estudos, ajudar no fórum — rende **XP** (experiência) e **Sementes** (moeda virtual do app). O aluno evolui por uma progressão de **5 animais** (Coruja → Raposa → Gavião → Leopardo → Tubarão), cada um dividido em 4 subfases (Filhote, Jovem, Adulto, Veterano) — 20 estágios ao todo, cada um com um conceito relacionado a hábitos de estudo (ex.: a Coruja representa "busca pelo saber, observação silenciosa"). Há também um mecanismo de incentivo à constância: ficar muitos dias sem abrir o app faz o XP diminuir aos poucos, o que estimula o retorno frequente.

RECOMPENSA

Loja de Cosméticos

Com as Sementes acumuladas, o aluno compra itens puramente estéticos para personalizar seu avatar — chapéus, acessórios e molduras de foto. Alguns itens são "de prestígio": só ficam disponíveis depois que o aluno atinge um determinado estágio de evolução, criando um objetivo de longo prazo.

COMPARAÇÃO ENTRE ALUNOS

Leaderboard (Ranking)

Mostra a classificação dos alunos por XP acumulado, reforçando o incentivo à participação através de comparação saudável entre colegas.

Identidade e configurações

PERFIL

Perfil e Perfil Público

O Perfil reúne as informações do próprio aluno: foto, bio, nível de gamificação e medalhas conquistadas. O Perfil Público mostra essas mesmas informações do ponto de vista de um colega, incluindo as avaliações recebidas como "professor" em matches já concluídos.

PREFERÊNCIAS

Configurações

Permite personalizar aparência (modo escuro, tamanho da fonte), notificações (quais tipos receber, som, vibração) e privacidade (quem pode ver o perfil, se o status "online" fica visível).

Gestão e suporte

GESTÃO DE CONTEÚDO

Painel Administrativo

Uma área restrita para gerenciar os posts do mural (por categoria: destaques, oportunidades, olimpíadas, avisos, passeios) e os usuários cadastrados (aprovar, suspender, alterar o tipo de conta), com uma visão geral em gráficos — útil para quem administra o conteúdo acompanhar o uso do app sem precisar mexer diretamente no banco de dados.

SUPORTE

Ajuda e Contato

Duas telas de apoio: uma de perguntas frequentes sobre o uso do app, e outra com os canais oficiais de contato das Escolas Padre Anchieta (WhatsApp, telefone, e-mail, site, endereço e agendamento de visita).

SEÇÃO 6

O Que Fica Guardado — Panorama dos Dados

Sem entrar em detalhes de banco de dados, vale um panorama de que tipo de informação o app guarda no Firestore e para qual finalidade — útil caso surjam perguntas sobre privacidade ou volume de dados na apresentação.

Grupo de dados	O que guarda
Usuários	Dados de cadastro, status da conta, progresso de gamificação (XP, moedas, streak), preferências sincronizadas, itens cosméticos comprados/equipados e avaliações recebidas.
Posts	Conteúdo do mural/feed publicado para os alunos.
Conversas e mensagens	Threads de chat entre pares de alunos e o conteúdo de cada mensagem.
Solicitações de match	Pedidos de pareamento entre alunos e o status de cada um (pendente, aceito, etc.).
Eventos escolares	Provas, feriados, avisos e eventos gerais cadastrados pela administração, organizados por série.
Lembretes	Lembretes pessoais que cada aluno cria para si mesmo no calendário.
Medalhas	Conquistas/badges associadas ao perfil de cada aluno.

Este documento não encontrou, dentro da pasta do projeto, um arquivo de regras de segurança do Firestore (que definiria, por exemplo, quem pode ler ou alterar cada tipo de dado). Isso não significa necessariamente que elas não existam — regras desse tipo também podem estar configuradas diretamente no painel do Firebase, fora do repositório de código. É um ponto a confirmar, e naturalmente entra na próxima etapa, de revisão de segurança.

SEÇÃO 7

Resumo da Stack Tecnológica

Uma tabela única, para consulta rápida na hora de montar os slides.

Camada	Tecnologia
Plataforma do app	React Native + Expo (SDK 57) — Android, iOS e Web a partir de um único código
Linguagem	TypeScript (modo estrito)
Navegação	React Navigation (stack)
Banco de dados	Cloud Firestore (Firebase) — tempo real, NoSQL
Autenticação	Firebase Authentication — e-mail e senha, com verificação por e-mail
Armazenamento local	AsyncStorage (sessão e preferências no dispositivo)
Notificações	Expo Notifications (push local) + escutadores em tempo real do Firestore
Mídia (fotos/vídeos/documentos)	Expo Image Picker, com arquivos convertidos e guardados como texto (Base64) no Firestore
Gráficos	React Native Chart Kit (painel administrativo)
Datas	date-fns
Identidade visual	Paleta de cores própria (com modo escuro) + fonte Poppins (Google Fonts)
Build e distribuição	EAS (Expo Application Services)

Leitura técnica gerada a partir do código-fonte do projeto "Symbius" (app JUMO), em 14 de setembro de 2026, como material de apoio para uma apresentação. Este documento descreve o que existe hoje no projeto; melhorias de segurança, desempenho e correção de bugs ficam para uma etapa seguinte.